Codex 통합 인수인계 (2026-11-09 월요일 전달)

작성일 2026-11-07. 조직 `ai-agent-orchestrator`의 저장소 4개에 걸친 변경을 한 문서로 합쳤습니다. 이 문서가 최신이며, 이전 단일 저장소 인수인계(범위 축소)는 3절에 흡수했습니다.

0. 먼저 읽기: 이 문서에서 가장 중요한 4가지

1. **이 저장소(legal-intake-fastapi-lab)는 접수·안내 Python action + 보안 action 9개 + 법률 마케팅 강화까지만** 개발합니다. LangChain·LangGraph·LangSmith·RAG·임베딩·벡터·memvid·Llama는 새 연구 저장소로 분리했습니다.
2. **마스킹과 입력 정책은 이 저장소가 기준(canonical)입니다.** 연구 저장소는 이 구현을 가져다 씁니다. 그런데 `mask_personal_data`, `check_input_policy`가 **아직 코드에 없습니다.** 월요일 최우선 구현 대상입니다.
3. **연구 저장소의 Equalizer는 설계 단계입니다.** 대표 결정 D1~D7이 끝나기 전에는 구현하지 않습니다(4절).
4. **README는 모두 main에 반영 완료.** 코드는 한 줄도 옮기거나 지우지 않았습니다. 이전은 월요일 이후 작업입니다.

1. 저장소 4개의 역할과 현재 상태

저장소	역할	main 상태 (2026-11-07)
`legal-intake-fastapi-lab` (이 저장소)	원래 접수 설계 + 보안 action 9개 + 마케팅 강화 (API 최대 11개). 마스킹의 기준	README·README.en·이 문서 반영. 코드 이전 전
`python-memvid-guardrail` → **`langchain-rag-research-agent`로 개명 예정**	재사용 검색·브리프 모듈(Edge Equalizer). 이 저장소에서 뺀 RAG 내용 + 기존 검색 설계 + 추가 요구사항	README v1.0(ko/en)·설계 검수·다이어그램(ko/en)·공부 가이드 PDF 반영. 구현 전
`spring-fastapi-async-refactor`	Spring Boot 제어 + FastAPI + memvid 하이브리드	**아직 변경 없음.** 하이브리드 내용이 아직 연구 저장소 `docs/legacy/`에 있음
`ai-multi-agent-orchestration`	허브. 멀티 에이전트 2개 소개	반영 완료 (planned → extension, Multi-agent 1·2 소개)

저장소 이름 변경은 대표님이 GitHub 설정에서 직접 합니다(이 세션은 설정 변경 권한이 없음). **이름이 바뀌기 전까지 코드와 문서의 `python-memvid-guardrail`은 그대로 둡니다.** 바뀐 뒤 링크를 일괄 수정합니다(6절 할 일 ⑦).

2. 발전 순서와 제약 (README 기준, 변경 없음)

- 순서: ① 이 저장소 → ③ 연구 저장소 → ② `spring-fastapi-async-refactor`.
- 앱 API 최대 11개: 접수 3 + `/health` 1 + 게시글 3 + 후기 2 + AI 마케팅 2. **검색 API는 추가하지 않습니다.** 게시글 수정·삭제, 독립 Chat/SSE, 단독 guardrail API 금지.
- 인증·인가, 자동 외부 게시, 일반 웹 검색, 운영 DB, Redis/Celery/Kafka 추가 금지. 기존 Java 서버·DB 수정 금지.
- `.env` 커밋 금지. fake/mock 성공과 실제 공급자 호출 성공은 구분해 기록.
- 보존 브랜치 `wip/codex-legal-snapshot`은 main에 합치지 않고, 그 안의 실패한 마케팅 검사 코드를 완료본으로 쓰지 않습니다.
- 완료 기준은 "구현·연결·시험 통과"입니다. 계획 문서나 샘플 실행 성공은 완료가 아닙니다.

3. 이 저장소: 범위 축소 (기존 인수인계 내용)

- 필수 범위: 접수·안내 Python action + 보안 action 9개 + 법률 마케팅 강화.
- 옛 RAG 절을 README에서 삭제하고, 7절 "RAG·검색 기능 분리"에 이전 후보 경로를 적었습니다.

- 이전 후보: `app/rag/`, `rag-runtime/`, RAG 관련 `scripts/`(E5·memvid·검색 평가), `fixtures/rag/`, `tests/rag/`, `tests/test_rag_b03_b05.py`, `docs/RAG_*.md`, `docs/CLAUDE_RAG_DESIGN_REVIEW.md`, `docs/EMBEDDING_RETRIEVAL_STUDY.md`, `docs/evidence/`.
- `app/rag` 바깥 앱 코드는 `app.rag`를 import하지 않는 것으로 확인했습니다. 삭제 전에 한 번 더 검색하세요. `tests/conftest.py`의 `rag_memvid`, `rag_lc` 마커도 함께 정리합니다.
- **이전은 복사 → 연구 저장소에서 검증 → 이 저장소에서 제거** 순서입니다. `uv run pytest -q -m "not nemo"`가 전후 모두 통과해야 합니다.

보안 action 9개 현황 (2026-11-07 `app/` 이름 검색 재확인)

action	상태
`validate_request_context`	`app/security/session_actions.py` 구현, `session_guard.py` 호출
`check_session_risk`	구현, 호출 확인
`record_risk_event`	구현, 호출 확인
`check_generation_admission`	action 이름으로 없음. `AdmissionManager`(`admission.py`)는 존재
`check_request_budget`	action 이름으로 없음. `BudgetStore`(`budget_store.py`)는 존재
`check_input_policy`	**`app/`에 없음** ← 연구 저장소가 의존
`mask_personal_data`	**`app/`에 없음** ← 연구 저장소가 의존
`check_output_policy`	`app/`에 없음
`check_marketing_response`	`app/`에 없음

이름 검색 결과이며 시험 통과 여부는 확인하지 않았습니다. 다른 이름으로 구현돼 있을 수 있으니 작업 전에 재확인하세요.

4. 연구 저장소 (langchain-rag-research-agent): 설계 내용과 대표 결정 대기

설계 요약 (자세한 근거는 연구 저장소 `docs/DESIGN_REVIEW.ko.md`)

- **Edge Equalizer(가칭)**: 독립 노브 2개. ① 창 = 유사도 임계값(High 정밀 / Low 넓은 탐색), ② 방패 = 골든룰 엣지 보존 가중치 `w`(High 추출 전용·클리셰 삭감 / Low 읽기 쉬운 요약). `최종 점수 = (1−w)·리랭크 관련도 + w·엣지 점수`. 프리셋: 소송·증거(High,High), 지식 사업·리서치(Low,High), 법률 검색(High,Mid 제안), 일반 요약(Low,Low 제안).
- 파이프라인: Guard_Input → Chunk_Ingest → Retrieve / Protected_Pick → Edge_Filter → Token_Check ⇄ Token_Diet → Final_Synthesize → Output_Check. 근거가 없으면 생성 호출 0회. 보호 항목만 예산을 넘으면 `BUDGET_PROTECTED_OVERFLOW`로 멈추고 자르지 않습니다.
- 온톨로지 5분류(Core_Hypothesis, Tech_Architecture, Market_Positioning, Constraint_Guardrail, Action_Item + 내부용 Unclassified). 파서는 위치만 표시하고 글을 고치지 않습니다.
- 하이브리드 검색: BM25/TF-IDF + 로컬 E5 벡터를 RRF로 합치고 상위 30개를 로컬 리랭커(BGE 기본). **유사도 임계값은 0.85 고정이 아니라 모델별 골든셋 보정**입니다.
- 보안: 로컬 기본, 외부 공급자는 인터페이스 뒤에서 실행 단위로만 허용(소송 프리셋 금지). LangSmith 추적은 서비스 코드에서 꺼두고 합성·마스킹 자료 평가에만 씁니다. 공개 저장소에는 합성 자료만 둡니다.
- **실측(합성 10문서/30청크, E5+memvid)**: 비교 세트 1위 적중 키워드 12/14 대 벡터 14/14, 확인 세트 5/6 대 6/6. **시제품(가짜 임베더)** 회수율 혼합 0.943. 실제 Llama·Qwen 요약 품질은 측정한 적이 없습니다.

대표 결정 (아직 답 없음. 답 전에는 해당 구현을 시작하지 않음)

#	결정	추천안
D1	외부 공급자 정책	A: 로컬 기본 + 실행 단위 명시 허용, 소송·상담은 금지
D2	Python import 이름	A: `rag_research_agent`
D3	마스킹 공유 방식	A: 이 저장소의 보안 모듈을 설치 가능한 하위 패키지로 분리해 의존
D4	Token Diet 감량 순서	리서치: Unclassified → Market → Tech → Action → Core. 소송·상담 순서는 대표 지정
D5	실제 자료 처리	A: 공개 저장소는 합성만, 실제 자료는 로컬 폴더
D6	legal-intake 연결 시점	A: 과제 완료 후 라이브러리로 연결, API 추가 없음
D7	Qwen을 비교 후보에 넣을지	대표 결정 (현재 방침은 기본 제외)

D3이 이 저장소에 직접 영향을 줍니다: **A로 정해지면 보안 모듈(마스킹·입력 정책)을 패키지로 분리**할 수 있게 구조를 잡아야 합니다. 정해지기 전에는 `app/security/` 구조를 크게 바꾸지 마세요.

5. 마스킹·입력 정책 공유 규칙

- 기준 구현은 이 저장소 `mask_personal_data`, `check_input_policy`입니다. 연구 저장소의 `guard/masking.py`는 **동결**(학습용 패턴 3개: 주민번호·이메일·전화)이고, 이 저장소 구현이 준비되면 교체합니다.
- 연구 저장소는 프로토콜(인터페이스)과 시험용 가짜 구현만 갖습니다. 이 저장소의 시그니처를 바꾸면 연구 저장소 프로토콜 영향도 같이 확인하세요.
- 마스킹은 저장·임베딩·trace 전에 적용됩니다.

6. 월요일에 할 일 (순서대로)

1. **이름 변경 확인.** 대표님이 `python-memvid-guardrail` → `langchain-rag-research-agent` 개명을 했는지 확인합니다. 안 했으면 ⑦은 보류.
2. **보안 action 현황 재확인** (3절 표를 코드와 대조, README 2절 상태 정정).
3. **`check_input_policy`, `mask_personal_data` 구현·시험.** 연구 저장소가 의존하므로 최우선. 함수 단위 시험과 실제 NeMo rail 시험을 구분합니다. 이어서 `check_output_policy`, 용량·예산 action 래핑(`check_generation_admission`, `check_request_budget`).
4. **서비스 경계**: `용량 확보 → 예산 예약 → 호출 → 정산 → 슬롯 반환`. timeout·취소·실패에서도 정리를 보장하고 시험.
5. **게시판·후기 → M01 요약 / M02 답변 초안.** 전체 출력 검사 후 `check_marketing_response`를 통과시키고, 최종 마스킹·문맥 확인 뒤에만 반환. 자동 게시 없음.
6. **RAG 코드 이전**(3절 이전 후보): 연구 저장소의 M0~M1(패키지 뼈대·검색, B02b 재현)이 준비된 뒤에 복사 → 검증 → 제거. 대표 D2(import 이름)가 정해진 뒤가 좋습니다.
7. **개명 후 링크 정리**: 이 저장소 README·README.en·문서의 `python-memvid-guardrail`을 새 이름으로 수정.
8. **하이브리드 내용 이전**(별도 작업): 연구 저장소 `docs/legacy/`의 하이브리드 README 2개와 `docs/hybrid-contract-design.ko.md`, `docs/study-guide.ko.md`, 하이브리드 PDF 3개, 아키텍처·보안·비용·Python 실행 다이어그램을 `spring-fastapi-async-refactor`의 `docs/hybrid/`로 옮깁니다. 새 브랜치에서 하고 main에는 PR로 합칩니다. 이전 후 연구 저장소에서 제거.

7. 연구 저장소 구현 순서 (D1~D7 결정 뒤)

M0~M1 패키지 뼈대·검색, B02b 재현 → M2 골든셋 임베딩 모델 비교 → M3~M4 공유 보안 연결·그래프·추적·외부 전송 0 시험 → **M5 Equalizer 핵심(온톨로지 파서, 노브 2개, 엣지 점수, 보호 후보 경로, Token Diet)을 실제 로컬 모델로 평가** → M6 리서치 툴

그림자 비교 → M7 공개·합성 자료로 법률 검색 시범. 측정은 항상 골든 엣지 회수율(n/총수), 보호 항목 탈락 수(0), 원문에 없는 숫자·ID 수(0), 인용 ID ⊆ 입력 ID로 하고, "유실 0" 같은 단언은 쓰지 않습니다.

8. 하지 말 것

- D1~D7 전에 Equalizer 구현 시작, 임계값 0.85 고정, 외부 임베딩·리랭커·원격 trace 기본 활성화.
- 공개 저장소에 실제 사건·개인정보·환자 기록 투입. 합성 자료만.
- 법률 판단·진단·치료 권고를 생성하는 기능 추가. 판례·상담 기록 검색 활용은 법률·의료 검토 전제의 후속 작업입니다(이 저장소에 검색 API를 넣지 않음).
- 증거 없는 "완성" 표기. 시제품(가짜 임베더) 수치를 실측처럼 쓰지 않기.
- Qwen 계열 기본 포함(D7 전).

9. 확인하지 않은 것

- 어떤 저장소에서도 코드를 실행하거나 시험을 돌리지 않았습니다. README·문서·파일 구조·이름 검색만 봤습니다.
- 연구 저장소의 시제품(합쳐지지 않은 Draft PR #5)과 실제 코드 동작은 이 문서 작성자가 검증하지 않았습니다.
- `spring-fastapi-async-refactor`와 `nightcoder`는 읽기만 했고 수정하지 않았습니다. nightcoder의 "N8 planned" 표기는 그대로입니다.
- 이전 후보 경로의 실제 파일 목록은 이전 시점에 다시 확인해야 합니다.